KONYA FOOD AND AGRICULTURE UNIVERSITY

2025-2026 ACADEMIC YEAR

Spring Semester

Faculty of Engineering and Architecture

Department of Computer Engineering

COMP4244-Microservice Based Software Design and Development

Declarative Web Service Client

Project 11 – Group 5

Rabia Leyla Kıyak - 212010020010

Ömer Torus – 222010020002

Furkan Özyücel - 222010020108

Abstract

This project implements a microservice-based Music Playlist Application to demonstrate declarative service-to-service communication with Spring Cloud OpenFeign. The system is divided into three Spring Boot microservices: User Service, Song Service, and Playlist Service. The Playlist Service is the main orchestration service. It uses Feign client interfaces to request user information and favorite genres from User Service, then requests songs by genre from Song Service, and finally returns a generated playlist response. The project also includes Docker Compose support with PostgreSQL so the services can run in a containerized environment.

1. Introduction

Microservice architecture separates an application into smaller services, where each service is responsible for a specific business capability. This separation improves modularity, but it also creates a communication problem because services must call each other over the network. In this project, the communication problem is demonstrated with a music playlist generation scenario.

The aim of the project is to show how a service can communicate with other services without manually writing repetitive HTTP request code. OpenFeign is selected because it allows remote HTTP APIs to be represented as Java interfaces with Spring annotations. This approach keeps the code readable and supports a clean service-to-service communication structure.

2. Background

2.1 Microservice Communication Problem

In monolithic applications, modules usually communicate within the same process. In microservice systems, each component may run as a separate application and communicate through HTTP. Manual approaches require developers to construct URLs, send requests, parse responses, and handle errors repeatedly. This increases boilerplate code and maintenance complexity.

2.2 Declarative Clients and OpenFeign

OpenFeign is a declarative HTTP client. Instead of writing manual HTTP calls, the developer defines an interface and annotates methods with mappings such as @GetMapping. OpenFeign generates the implementation automatically at runtime. In this project, Playlist Service enables Feign support with @EnableFeignClients and uses separate UserClient and SongClient interfaces.

2.3 Compared Approaches

Client	Style	Main Advantage	Limitation
RestTemplate	Imperative	Simple and familiar for direct HTTP calls.	Requires more manual URL, request, and response handling.
WebClient	Reactive	Useful for asynchronous and high-traffic systems.	More complex for a small demonstration project.
OpenFeign	Declarative	Readable interfaces with less code and Spring MVC compatibility.	Requires correct service URLs and Feign configuration.

3. Development Details

3.1 System Overview

The system contains three microservices and one PostgreSQL database container. Docker Compose starts all services together. User Service runs on port 8081, Song Service runs on port 8082, Playlist Service runs on port 8083, and PostgreSQL is exposed on port 5432.

Microservice	Port	Responsibility	Role in Scenario
User Service	8081	Stores user profile information and favorite genres.	Provides user details and favorite genre data.
Song Service	8082	Stores songs and supports genre-based song filtering.	Returns songs that match requested genres.
Playlist Service	8083	Creates playlists by combining user and song data.	Calls User Service and Song Service through OpenFeign.
PostgreSQL	5432	Database container used by the services.	Provides persistence through JDBC connection settings.

3.2 Docker and Runtime Environment

The docker-compose.yml file defines postgres, user-service, song-service, and playlist-service containers. Each microservice is built from its own service folder and Dockerfile. The services depend on the PostgreSQL health check, which helps ensure that the database is ready before the application containers start. The Spring datasource URL is configured as jdbc:postgresql://postgres:5432/postgres inside the Docker network.

```
user-service:     localhost:8081
song-service:     localhost:8082
playlist-service: localhost:8083
postgres:         localhost:5432
```

3.3 OpenFeign Implementation

OpenFeign is used inside Playlist Service. The main application class contains @EnableFeignClients, which activates scanning for Feign client interfaces. The clients are configured to call services by their Docker Compose service names. This is important because inside the Docker network, service names such as user-service and song-service can be used as host names.

```
@FeignClient(name = "user-service", url = "http://user-service:8081", configuration =
FeignConfig.class)
public interface UserClient {
    @GetMapping("/users/{id}")
    UserDTO getUserById(@PathVariable("id") Long id);

    @GetMapping("/users/{id}/favorites")
    List<String> getFavoriteGenres(@PathVariable("id") Long id);
}

@FeignClient(name = "song-service", url = "http://song-service:8082", configuration =
FeignConfig.class)
public interface SongClient {
    @GetMapping("/songs/genre/{genre}")
```

```
    List<SongDTO> getSongsByGenre(@PathVariable("genre") String genre);
}
```

3.4 Playlist Service API

The Playlist Controller exposes two endpoints. The most important endpoint is the POST endpoint that triggers automatic playlist generation for a selected user. The second endpoint lists playlists that belong to a user.

Method	Endpoint	Purpose
POST	/playlists/generate/{userId}	Generates an automatic playlist for the given user by using Feign calls.
GET	/playlists/user/{userId}	Returns playlist records belonging to the given user.

3.5 Data Transfer Objects

The communication between services uses DTO classes. UserDTO contains id, username, email, and favoriteGenres. SongDTO contains id, title, artist, genre, and durationSeconds. PlaylistResponseDTO returns playlistId, playlistName, owner, and a list of SongDTO objects. This keeps the response structure clear and avoids exposing unnecessary internal entity details.

```
PlaylistResponseDTO
- Long playlistId
- String playlistName
- String owner
- List<SongDTO> songs

UserDTO
- Long id
- String username
- String email
- List<String> favoriteGenres

SongDTO
- Long id
- String title
- String artist
- String genre
- int durationSeconds
```

3.6 Communication Flow

1. Client sends POST /playlists/generate/{userId} to Playlist Service on port 8083.
2. Playlist Service calls User Service through UserClient using /users/{id}.
3. Playlist Service calls User Service again through /users/{id}/favorites to retrieve favorite genres.
4. For each genre, Playlist Service calls Song Service through SongClient using /songs/genre/{genre}.
5. Playlist Service combines the returned user and song data, saves the playlist, and returns PlaylistResponseDTO as JSON.

4. Result and Discussion

The project successfully demonstrates declarative web service communication in a microservice architecture. OpenFeign reduces the amount of manual HTTP client code and makes the Playlist Service easier to read. The project also satisfies the requirement of having at least three microservices running in a Docker environment. Because the services are separated, each service has a clear responsibility, while the Playlist Service demonstrates how microservices can cooperate to complete a business operation.

5. Conclusion

This project shows how OpenFeign can be used to simplify communication between Spring Boot microservices. The Music Playlist Application contains User Service, Song Service, and Playlist Service. The Playlist Service acts as the coordinator and uses declarative Feign clients to retrieve user preferences and songs by genre. Docker Compose provides a reproducible way to run the project and supports the final submission requirements.